



**T.C.
FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ**

**Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü**

İŞLETİM SİSTEMLERİ DÖNEM PROJESİ

ProcX - Gelişmiş Süreç Yönetim Sistemi

**MUHAMMED EMİN KORKUNÇ
2021221054**

Proje Danışmanı: Dr.Öğr.Üyesi Samet KAYA,
Arş.Gör. Zeliha Kaya AKÇELİK

İstanbul, [Aralık 2025]

1. ÖZET	1
2. PROJE TANIMI VE HEDEFLER	1
3. SİSTEM MİMARİSİ	2
3.1. Veri Yapıları	2
3.2. IPC Mekanizmaları	2
3.3. Thread Yapısı	3
4. FONKSİYONEL GEREKSİNİMLER	3
4.1. Process Başlatma (fork + exec)	3
4.2. Process Listeleme	4
4.3. Process Sonlandırma	5
4.4. Monitor Thread	6
4.5. IPC Listener Thread	6
4.6. Çıkış ve Temizlik	7
5. TEST SENARYOLARI VE EKRAN ÇIKTILARI	7
5.1. Test 1: Tek Instance - Process Başlatma ve Listeleme	7
5.2. Test 2: Çoklu Instance - IPC Testi	8
5.4. Test 4: Process Sonlandırma	9
5.5. Test 5: Monitor Thread Testi	10
6. KAYNAK KODLAR	10
8. SONUÇ VE KAZANIMLAR	11
9. EK : KULLANILAN TEMEL SİSTEM ÇAĞRILARI	11

1. ÖZET

Bu proje kapsamında, Linux işletim sistemi üzerinde çalışan, çoklu işlem (multiprocessing) ve süreçler arası iletişim (IPC) temellerine dayanan **ProcX** isimli bir süreç yönetim sistemi geliştirilmiştir. Proje, kullanıcıların terminal üzerinden programları "Attached" (bağlı) veya "Detached" (bağımsız) modda çalıştırmasına, bu süreçleri listelemesine ve sonlandırmasına olanak tanır. Birden fazla terminalde çalışan ProcX örnekleri, Paylaşılan Bellek (Shared Memory) ve Mesaj Kuyrukları (Message Queue) kullanarak birbirleriyle senkronize olur ve anlık veri paylaşımı yapar.

2. PROJE TANIMI VE HEDEFLER

ProcX, nohup benzeri bir yapı sunarak, bir terminal kapansa bile arka planda çalışan süreçlerin (Detached mode) hayatta kalmasını sağlayan, aynı zamanda aktif çalışan süreçlerin merkezi bir havuzda yönetilmesine imkan tanıyan bir sistemdir.

Projenin Temel Hedefleri:

- **Process Yönetimi:** fork() ve execvp() sistem çağrıları ile yeni süreçlerin oluşturulması.

- **Thread Programlama:** Ana akışın yanı sıra arka planda çalışan Monitor ve Listener iş parçacıklarının (threads) yönetimi.
- **IPC (Inter-Process Communication):** Süreçler arası veri paylaşımı ve haberleşme.
- **Senkronizasyon:** Semaphore kullanılarak veri bütünlüğünün korunması (Race Condition önleme).
- **Sinyal Yönetimi:** Süreçlerin kontrollü bir şekilde sonlandırılması.

3. SİSTEM MİMARİSİ

Proje, POSIX standartlarına uygun olarak C dili ile geliştirilmiştir. Sistem mimarisi üç ana bileşenden oluşur: Veri Yapıları, IPC Mekanizmaları ve Thread Yapısı.

3.1. Veri Yapıları

Proje gereksinimlerine (PDF) birebir sadık kalınarak aşağıdaki veri yapıları kullanılmıştır:

- **ProcessInfo:** Her bir sürecin kimlik, komut, mod, durum ve sahiplik bilgilerini tutar.
- **SharedData:** Tüm ProcX örneklerinin erişebildiği, maksimum 50 süreci barındıran ortak bellek alanıdır.
- **Message:** Instance'lar arası iletişimi sağlayan mesaj yapısıdır.

3.2. IPC Mekanizmaları

Sistemde üç farklı IPC yöntemi entegre çalışmaktadır:

1. **Shared Memory (/procx_shm_area):** Süreç listesinin (Process Table) tüm terminaller tarafından ortaklaşa okunup yazılmasını sağlar.
2. **Semaphore (/procx_sem_lock):** Paylaşılan belleğe aynı anda birden fazla yazma işleminin yapılmasını engelleyerek veri bütünlüğünü korur (Mutual Exclusion).
3. **Message Queue (System V):** Bir terminalde işlem yapıldığında (yeni süreç başlatma veya sonlandırma), diğer terminallere anlık bildirim ([IPC] mesajı) gönderilmesini sağlar.

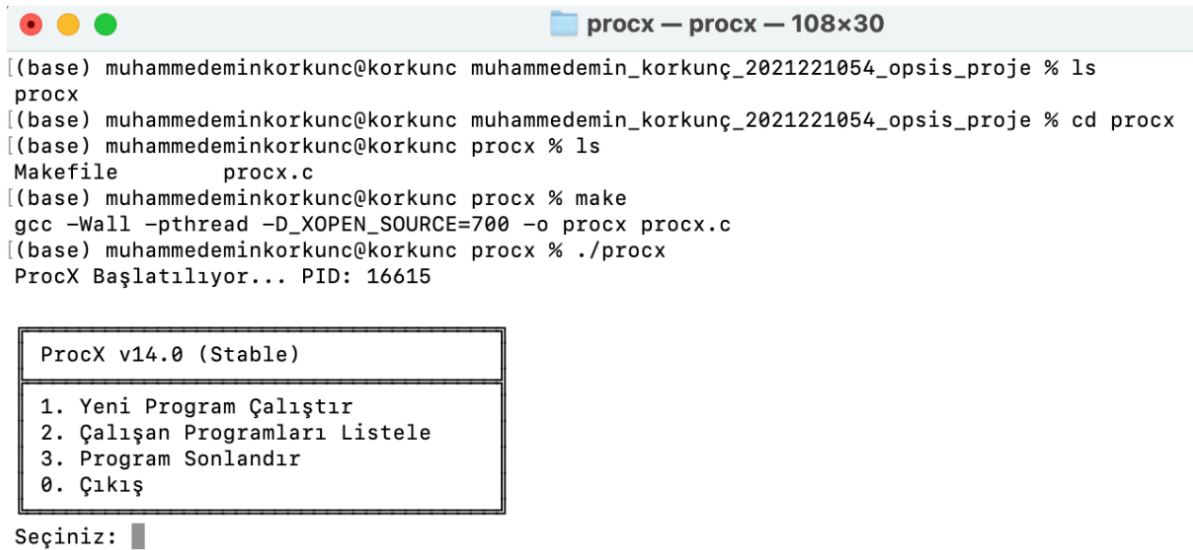
3.3. Thread Yapısı

Her çalışan ProcX programı (instance) bünyesinde 3 adet thread barındırır:

1. **Main Thread:** Kullanıcı menüsünü gösterir ve girdileri işler.
2. **Monitor Thread:** Her 2 saniyede bir çalışarak süreçlerin durumunu (waitpid) kontrol eder. Biten süreçleri bellekten temizler.
3. **IPC Listener Thread:** Mesaj kuyruğunu sürekli dinler ve diğer terminallerden gelen bildirimleri ekrana yansıtır.

4. FONKSİYONEL GEREKSİNİMLER

Proje, belirtilen tüm fonksiyonel gereksinimleri başarıyla yerine getirmektedir.



```
procx — procx — 108x30
[(base) muhammedeminkorkunc@korkunc muhammedemin_korkunc_2021221054_opsis_proje % ls
procx
[(base) muhammedeminkorkunc@korkunc muhammedemin_korkunc_2021221054_opsis_proje % cd procx
[(base) muhammedeminkorkunc@korkunc procx % ls
Makefile      procx.c
[(base) muhammedeminkorkunc@korkunc procx % make
gcc -Wall -pthread -D_XOPEN_SOURCE=700 -o procx procx.c
[(base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 16615

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: █
```

TEST-1 : Programın Makefile ile başlatılması işlemi

4.1. Process Başlatma (fork + exec)

Kullanıcıdan alınan komut, `fork()` ile oluşturulan çocuk süreçte `execvp()` kullanılarak çalıştırılır.

- **Attached Mod (0):** Süreç terminale bağlı çalışır.
- **Detached Mod (1):** `setsid()` çağırısı ile yeni bir oturum oluşturulur, böylece terminal kapansa bile süreç çalışmaya devam eder.

```

(base) muhammedeminkorkunc@korkunc muhammedemin_korkunc_2021221054_opsis_proje % cd procx
(base) muhammedeminkorkunc@korkunc procx % ls
Makefile      procx.c
(base) muhammedeminkorkunc@korkunc procx % make
gcc -Wall -pthread -D_XOPEN_SOURCE=700 -o procx procx.c
(base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 16615

```

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 100
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 20651

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: █

TEST-2 : 6.1. Process Başlatma (fork + exec)

4.2. Process Listeleme

Kullanıcı 2 seçeneğini seçtiğinde, program Semaphore kilidini alarak Shared Memory'yi okur. Aktif süreçler; PID, Komut, Mod, Sahip PID ve Çalışma Süresi sütunları ile tablo halinde listelenir.

procx — procx — 108x41

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 100
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 20651

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 2

ÇALIŞAN PROGRAMLAR				
PID	KOMUT	MOD	SAHIP	SURE
20651	sleep 100	Detached	16615	81s

Toplam: 1 process

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[MONITOR] Process 20651 sonlandı
Seçiniz: █

TEST-3 : 6.2. Process Listeleme

4.3. Process Sonlandırma

Kullanıcıdan alınan PID'ye kill(pid, SIGTERM) sinyali gönderilir. Sürecin listeden silinmesi işlemi, sinyali takiben Monitor Thread tarafından otomatik olarak yapılır.

```
procx — procx — 108x57
((base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 23323

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 100
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 23459

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 3
Sonlandırılacak PID: 23459
[INFO] Process 23459'e SIGTERM gönderildi

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[MONITOR] Process 23459 sonlandı
Seçiniz: 2



| ÇALIŞAN PROGRAMLAR |       |     |       |      |
|--------------------|-------|-----|-------|------|
| PID                | KOMUT | MOD | SAHIP | SURE |
|                    |       |     |       |      |



Toplam: 0 process

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: █
```

TEST-4 : 6.3. Process Sonlandırma

4.4. Monitor Thread

Arka planda çalışan bu thread, "Zombie Process" oluşumunu engeller. Süresi dolan veya dışarıdan sonlandırılan süreçleri tespit eder, hafızadan temizler ve kullanıcıya [MONITOR] bildirimi yapar.

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 3
Sonlandırılacak PID: 23459
[INFO] Process 23459'e SIGTERM gönderildi

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[MONITOR] Process 23459 sonlandı
Seçiniz: 2

TEST-5 : 6.4. Monitor Thread

4.5. IPC Listener Thread

Diğer ProcX örneklerinden gelen CMD_START ve CMD_TERMINATE mesajlarını yakalar. Kendi gönderdiği mesajları filtreler, diğerlerinden gelenleri ekrana basar.

procx — procx — sleep — 108x29

((base) muhammedminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 26207

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 100
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 26317

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:

procx_deneme — procx — 109x27

((base) muhammedminkorkunc@korkunc procx_deneme % ./procx
ProcX Başlatılıyor... PID: 26235

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[IPC] Yeni process başlatıldı: PID 26317
Seçiniz: 2

ÇALIŞAN PROGRAMLAR				
PID	KOMUT	MOD	SAHIP	SURE
26317	sleep 100	Detached	26207	3s

Toplam: 1 process

TEST-6 : 6.5. IPC Listener Thread

4.6. Çıkış ve Temizlik

Programdan çıkış (0) yapıldığında:

- **Attached** süreçler sonlandırılır.
- **Detached** süreçler korunur (Persistence).
- Son çalışan instance kapandığında, Shared Memory ve Message Queue kaynakları sistemden temizlenir (unlink, msgctl).

Seçiniz: 2

ÇALIŞAN PROGRAMLAR				
PID	KOMUT	MOD	SAHIP	SURE
28465	sleep 100	Detached	28383	5s

Toplam: 1 process

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 0
[SİSTEM] Detached süreçler nedeniyle hafıza korundu.
ProcX kapatıldı.

TEST-7 : 6.6. Çıkış ve Temizlik

5. TEST SENARYOLARI VE EKRAN ÇIKTILARI

Projenin doğruluğu aşağıdaki senaryolarla test edilmiştir.

5.1. Test 1: Tek Instance - Process Başlatma ve Listeleme

Programın derlenmesi (Makefile), başlatılması ve tek bir terminal üzerinde süreç oluşturup listeleme yeteneği test edilmiştir.

```
procx --zsh -- 108x54
((base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 30182

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 100
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 30237

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 2

ÇALIŞAN PROGRAMLAR
PID  KOMUT  MOD  SAHIP  SURE
30237  sleep 100 Detached 30182  4s

Toplam: 1 process

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 0
[SİSTEM] Detached süreçler nedeniyle hafıza korundu.
ProcX kapatıldı.
(base) muhammedeminkorkunc@korkunc procx %

procx --zsh -- 108x54
((base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 31134

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 2

ÇALIŞAN PROGRAMLAR
PID  KOMUT  MOD  SAHIP  SURE
30237  sleep 100 Detached 30182  67s

Toplam: 1 process

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 
```

TEST-8 : 7.1. Test 1: Tek Instance - Process Başlatma ve Listeleme

5.2. Test 2: Çoklu Instance - IPC Testi

İki farklı terminal açılarak sistemin dağıtık yapısı test edilmiştir. Birinci terminalde başlatılan süreç, ikinci terminalde IPC mesajı olarak görüntülenmiş ve listede yer almıştır.

```
procx -- procx - sleep -- 108x49
((base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 33701

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 200
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 33784

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:

procx_deneme -- procx -- 109x49
((base) muhammedeminkorkunc@korkunc procx_deneme % ./procx
ProcX Başlatılıyor... PID: 33729

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[IPC] Yeni process başlatıldı: PID 33784
Seçiniz: 2

ÇALIŞAN PROGRAMLAR
PID  KOMUT  MOD  SAHIP  SURE
33784  sleep 200 Detached 33701  7s

Toplam: 1 process

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 
```

TEST-9 : 7.2. Test 2: Çoklu Instance - IPC Testi

5.3. Test 3: Attached vs Detached Modu

Bu kritik testte, program kapatıldığında Attached sürecin öldüğü, Detached sürecin ise arka planda yaşamaya devam ettiği (Persistence) doğrulanmıştır. ps aux komutu ile sistem seviyesinde doğrulama yapılmıştır.

```
procx -- zsh -- 108x49
(base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 35613

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 300
Mod seçin (0: Attached, 1: Detached): 0
[SUCCESS] Process başlatıldı: PID 35749

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 400
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 35833

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 0
[SİSTEM] Detached süreçler nedeniyle hafıza korundu.
ProcX kapatıldı.
(base) muhammedeminkorkunc@korkunc procx %

procx_deneme -- procx -- 109x49
(base) muhammedeminkorkunc@korkunc procx_deneme % ./procx
ProcX Başlatılıyor... PID: 36644

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 2

ÇALIŞAN PROGRAMLAR
PID      KOMUT      MOD      SAHIP      SURE
35833    sleep 400  Detached  35613      60s

Toplam: 1 process

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 
```

base) muhammedeminkorkunc@korkunc procx_deneme % ps aux

```
muhammedeminkorkunc 35833  0,0  0,0 408495776  432  ??  Ss   6:49PM  0:00.00 sleep 400
```

TEST-10 : 7.3. Test 3: Attached vs Detached Modu

5.4. Test 4: Process Sonlandırma

Kullanıcı tarafından manuel olarak bir sürecin sonlandırılması ve sistemin buna verdiği tepki test edilmiştir.

```
procx -- procx -- 108x49
(base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 39289

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 1
Çalıştırılacak komutu girin: sleep 500
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 39425

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz: 3
Sonlandırılacak PID: 39425
[INFO] Process 39425'e SIGTERM gönderildi

ProcX v14.0 (Stable)
1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

Seçiniz:
[MONITOR] Process 39425 sonlandı
```

TEST-11 : 7.4. Test 4: Process Sonlandırma

5.5. Test 5: Monitor Thread Testi

Kısa süreli (sleep 5) bir süreç başlatılarak, süresi dolduğunda Monitor Thread'in bunu otomatik olarak algılayıp temizlediği doğrulanmıştır.

```
(base) muhammedeminkorkunc@korkunc procx % ./procx
ProcX Başlatılıyor... PID: 40241
```

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

```
Seçiniz: 1
Çalıştırılacak komutu girin: sleep 5
Mod seçin (0: Attached, 1: Detached): 1
[SUCCESS] Process başlatıldı: PID 40350
```

ProcX v14.0 (Stable)

1. Yeni Program Çalıştır
2. Çalışan Programları Listele
3. Program Sonlandır
0. Çıkış

```
Seçiniz:
[MONITOR] Process 40350 sonlandı
Seçiniz: █
```

TEST-12 : 7.5. Test 5: Monitor Thread Testi

6. KAYNAK KODLAR

6.1. Makefile

6.2. procx.c

8. SONUÇ VE KAZANIMLAR

Bu proje kapsamında, UNIX tabanlı işletim sistemlerinde süreç yönetimi ve IPC mekanizmaları üzerine kapsamlı bir çalışma gerçekleştirilmiştir. Proje süresince aşağıdaki kritik noktalarda teknik derinlik sağlanmıştır:

1. **Senkronizasyon Sorunları:** Birden fazla terminalin aynı anda Shared Memory'ye erişmesi durumunda oluşabilecek "Race Condition" (Yarış Durumu), Semaphore mekanizması ile çözülmüş ve veri bütünlüğü garanti altına alınmıştır.
2. **Zombie Process Yönetimi:** Monitor Thread içerisinde kullanılan `waitpid(WNOHANG)` yapısı sayesinde, sonlanan süreçlerin sistem kaynaklarını işgal etmesi (zombie durumu) engellenmiş ve işletim sistemi kaynakları verimli kullanılmıştır.
3. **Kalıcılık (Persistence):** Programın kapatılıp açılması durumunda veri kaybını önlemek için Shared Memory'nin yaşam döngüsü programdan bağımsız hale getirilmiş, ancak "Memory Leak" oluşmaması için son kullanıcı çıktığında temizlik yapan bir Cleanup mekanizması kurulmuştur.
4. **Güvenlik:** Kullanıcıdan alınan girdilerde oluşabilecek "Buffer Overflow" (Tampun Taşması) risklerine karşı güvenli string fonksiyonları ve giriş doğrulama (input validation) algoritmaları geliştirilmiştir.

Sonuç olarak ProcX, sadece temel isterleri karşılayan bir ödev değil, gerçek dünya senaryolarında (elektrik kesintisi, hatalı giriş, çoklu erişim vb.) kararlı çalışabilen dayanıklı (robust) bir sistem olarak tasarlanmıştır.

9. EK : KULLANILAN TEMEL SİSTEM ÇAĞRILARI

Projede kullanılan kritik POSIX sistem çağrıları ve kullanım amaçları aşağıda özetlenmiştir:

Sistem Çağrısı	Kullanım Amacı	İlgili Gereksinim
<code>fork()</code>	Yeni bir çocuk süreç (child process) oluşturmak için.	6.1 Process Başlatma
<code>execvp()</code>	Oluşturulan süreçte yeni bir program (örn: sleep) çalıştırmak için.	6.1 Process Başlatma

<code>setsid()</code>	Detached süreçleri terminalden bağımsız hale getirmek için.	6.1 Process Başlatma
<code>shm_open()</code> / <code>mmap()</code>	Süreç tablosunu tutan paylaşılan belleği oluşturmak ve erişmek için.	5.2 IPC Mekanizmaları
<code>sem_open()</code> / <code>sem_wait()</code>	Eşzamanlı erişimde veri bozulmasını önlemek (kilitleme) için.	5.2 IPC Mekanizmaları
<code>msgget()</code> / <code>msgsnd()</code>	Terminaller arası anlık bildirim göndermek için.	6.5 IPC Listener
<code>kill()</code>	Belirli bir sürece sinyal (SIGTERM) göndermek veya durumunu kontrol etmek (0) için.	6.3 Sonlandırma
<code>pthread_create()</code>	Arayüzü bloklamadan arka plan işlerini (Monitor, Listener) yürütmek için.	5.3 Thread Yapısı